Software Security Assignment

Introduction

This assignment includes:

1. Task 1 – SecureAI MathOracle 40 points
2. Task 2 – MindVault, the Memory Clinic 30 points
3. Task 3 – Secure Casino 30 points

Total: **100 points**.

Read each task carefully and exploit the program's vulnerabilities to complete it. We provide source code to help you understand the program, but it may differ slightly from the code used to build the executable. Use the supplied binaries for the attacks; do not rebuild them from the source code. Run them on the course server `workbench3.cs.hku.hk` or in your own x86-64 Linux environment. See `Tutorial_VM.pdf` for setup instructions.

Each task ships a `flag.txt` file next to the binary, because the program itself reads it when you succeed. Simply opening the flag file is not a solution: your report must demonstrate that you obtained the flag by exploiting the vulnerabilities of the program.

Submission

Please submit a report in PDF format to answer the questions listed in each task. Include screenshots wherever a question asks for them, and put your final exploit scripts (if any) in an appendix of the same PDF. The report must be your own work.

Environment and Tooling

Use `gdb` for debugging and `pwntools` for Python scripts that interact with the programs. Both are pre-installed on the course server. For setup instructions, see `Tutorial_VM.pdf`, section A.4 (server) or B.1 (your own Linux machine).

All tasks can be debugged with plain GDB. We recommend `pwndbg`, a GDB plugin that makes registers, stack frames and heap chunks easier to inspect. Commands such as `stack`, `telescope` and `vis-heap-chunks` require this plugin. With the `gdb` alias from `Tutorial_VM.pdf`, section A.4, the `gdb` commands below launch pwndbg directly. Without the alias, use `~/pwndbg/bin/pwndbg` in place of `gdb` for the portable version.

Run each binary from its own task directory so it can find `flag.txt`. Run your exploit script from that directory too.

Use `pwntools` to send and receive bytes, including non-printable characters:

```
from pwn import *
p = process("./mathoracle") # start the program
p.recvuntil(b"Your name: ") # wait for the prompt
p.sendline(b"Alice") # send bytes followed by a newline
```

Use `send()` for exact bytes and `sendline()` to add a newline. On x86-64, integers are stored in little-endian order:

```
payload = b"A" * 32 + p32(0) + p64(0x555555555000)
# p32(x): pack the integer x into 4 little-endian bytes
# p64(x): pack the integer x into 8 little-endian bytes (x86-64 addresses)
leaked = u64(p.recvn(8)) # decode exactly 8 raw bytes as an integer
```

This decoding example assumes the next eight bytes are raw data. For a text address printed by `%p`, use `int(text, 16)` instead.

GDB disables ASLR by default. Use it to inspect memory and measure offsets, but do not copy its concrete addresses or canary values into your exploit. Leak the values through the program and use them in the *same run*; values from a separate GDB session or an earlier run will not work.

Calling Conventions – x86-32 vs. x86-64

The lectures introduce 32-bit x86 cdecl, which passes arguments on the stack. This assignment uses Linux x86-64 (System V AMD64), which passes the first six integer or pointer arguments in registers. The basic ideas of stack frames and buffer overflows still apply, but addresses are eight bytes wide and the layouts differ.

This section compares the two conventions using integer arguments and introduces the debugger commands used in the assignment.

The example program

```
#include <stdio.h>

long foo(long a, long b, long c, long d, long e, long f, long g)
{
    return a + b + c + d + e + f + g;
}

int main(void)
{
    long r = foo(1, 2, 3, 4, 5, 6, 7);
    printf("foo returned %ld\n", r);
    return 0;
}
```

Save this example as `callconv.c`. Its seven integer arguments show both register and stack argument passing on x86-64.

```
gcc -g -O0 -fno-omit-frame-pointer -o callconv64 callconv.c
gcc -g -O0 -fno-omit-frame-pointer -m32 -o callconv32 callconv.c
```

The course server supports 32-bit compilation. Compile both versions there and follow the debugger steps yourself; the practice will help with the assignment.

Where the arguments live

On x86-32 cdecl, `main` places all seven arguments on the stack in reverse order: 7 first, 1 last. The `call` instruction then pushes the return address. At `foo`'s first instruction, `[esp]` holds the return address, `[esp+4]` holds argument 1, and the remaining arguments follow at increasing addresses, four bytes apart.

On x86-64, `main` puts arguments 1–6 in `RDI`, `RSI`, `RDX`, `RCX`, `R8` and `R9`, in that order. At `foo`'s first instruction, `[rsp]` holds the return address and `[rsp+8]` holds argument 7.

The first six arguments arrive in registers; the function may later copy them to the stack.

In these examples, the return value is in `EAX` (32-bit) or `RAX` (64-bit). After the function establishes its frame with `push ebp/rbp` and `mov ebp, esp` or `mov rbp, rsp`, arguments can also be located relative to the frame pointer, as shown in the table. Before that, the frame pointer still belongs to the caller.

	x86-32 (cdecl)	**x86-64 (System V AMD64)**
Arguments 1–6	on the stack, pushed right-to-left	in `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`
Arguments 7+	on the stack	on the stack, above the return address
After prologue	`[ebp+8]`, `[ebp+0xc]`, ...	registers, or `[rbp+0x10]`, ... for 7+
Return value	`EAX`	`RAX`
Cleanup	caller restores stack	caller restores stack (7+)
Callee-saved	`EBX`, `ESI`, `EDI`, `EBP`	`RBX`, `RBP`, `R12`–`R15`
Word size	4 bytes	8 bytes
Alignment	depends on the ABI	`RSP` 16-byte aligned before `call`

The session on x86-64

At the debugger 1 — start and stop.

```
gdb -q ./callconv64 # start gdb
break *foo # the star: stop at foo's FIRST machine instruction
run # run until the breakpoint hits
```

```
$ gdb -q ./callconv64

Reading symbols from ./callconv64...
pwndbg: loaded 179 pwndbg commands and 46 shell commands. Type pwndbg [--shell | --all] [filter .
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_ .
pwndbg> break *foo

Breakpoint 1 at 0x1149: file callconv.c, line 4.
pwndbg> run

Starting program: /tmp/ccshot/callconv64
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, foo (a=0, b=0, c=0, d=0, e=0, f=0, g=7) at callconv.c:4
4{
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
```

Figure 1: x86-64: the session starts and the breakpoint hits at `callconv.c:4`.

In Figure 1, GDB displays `a=0, ..., f=0, g=7`. In this build, the debug information refers to stack slots for the first six parameters, but those slots have not been initialized yet. Their actual incoming values are in the registers. GDB's parameter display at this early stop may differ between builds.

At the debugger 2 — the context panels.

Every time the program stops, `pwndbg` prints three panels by itself, without being asked: REGISTERS, DISASM and STACK. The next two figures are the first two panels of this very stop; read the prose under each before moving on.

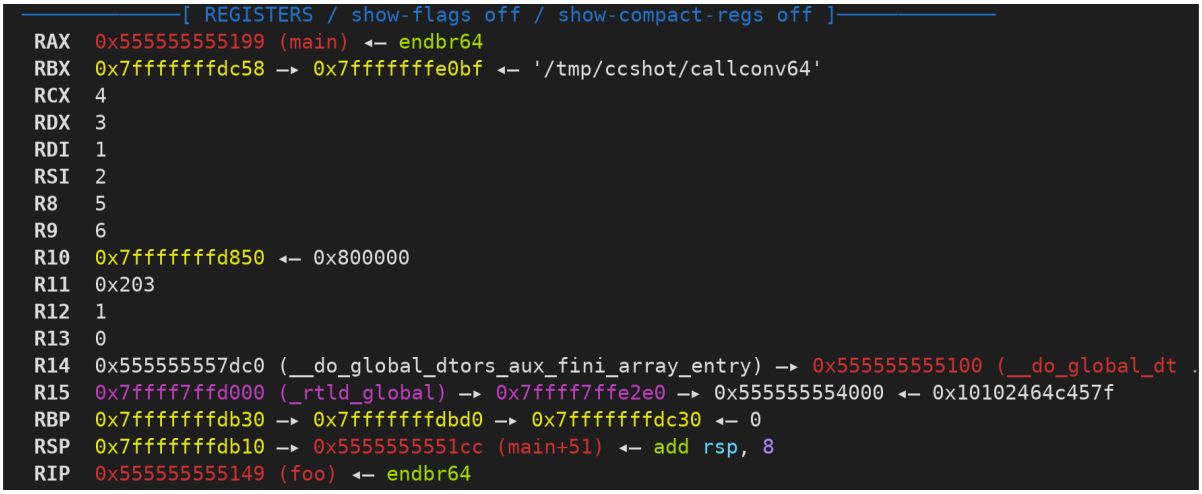


Figure 2: x86-64: the REGISTERS panel at the stop.

The REGISTERS panel shows the incoming arguments directly: `RDI`=1, `RSI`=2, `RDX`=3, `RCX`=4, `R8`=5 and `R9`=6. Use these registers to check arguments 1–6 at entry.

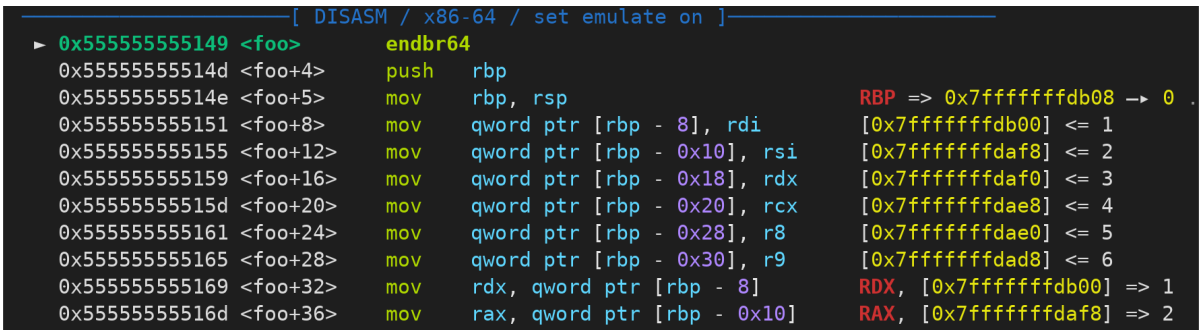


Figure 3: x86-64: the DISASM panel at the same stop.

The DISASM panel shows the function prologue. `push rbp; mov rbp, rsp` establishes the frame. Later, instructions such as `mov [rbp-8], rdi` copy the register arguments into stack slots; they do not remove the values from the registers. `break *foo` stops before this prologue. Use `si` to step one instruction at a time and observe the changes.

At the debugger 3 — dump the stack by hand.

```
stack 12 # show 12 pointer-sized words from the stack pointer
```

Learn this command well: it is the one you will use in Task 1 to measure where the canary and the return address sit.

```
pwndbg> stack 12

00:0000│ rsp 0x7fffffffdb10 —▸ 0x5555555551cc (main+51) ◂— add rsp, 8
01:0008│-018 0x7fffffffdb18 ◂— 7
02:0010│-010 0x7fffffffdb20 —▸ 0x7fffffffdc10 —▸ 0x555555555060 (_start) ◂— endbr64
03:0018│-008 0x7fffffffdb28 —▸ 0x7fffffffdc58 —▸ 0x7fffffffe0bf ◂— '/tmp/ccshot/callconv64'
04:0020│ rbp 0x7fffffffdb30 —▸ 0x7fffffffdbd0 —▸ 0x7fffffffdc30 ◂— 0
05:0028│+008 0x7fffffffdb38 —▸ 0x7ffff7c2a1ca (__libc_start_call_main+122) ◂— mov edi, eax
06:0030│+010 0x7fffffffdb40 —▸ 0x7fffffffdb80 —▸ 0x555555557dc0 (__do_global_dtors_aux_fini_arr .
07:0038│+018 0x7fffffffdb48 —▸ 0x7fffffffdc58 —▸ 0x7fffffffe0bf ◂— '/tmp/ccshot/callconv64'
08:0040│+020 0x7fffffffdb50 ◂— 0x155554040
09:0048│+028 0x7fffffffdb58 —▸ 0x555555555199 (main) ◂— endbr64
0a:0050│+030 0x7fffffffdb60 —▸ 0x7fffffffdc58 —▸ 0x7fffffffe0bf ◂— '/tmp/ccshot/callconv64'
0b:0058│+038 0x7fffffffdb68 ◂— 0x94773dd2f38043a9
```

Figure 4: x86-64: `stack 12` at the same stop.

In Figure 4, word 0 at `[rsp]` is the return address (`main+51`, where `main` resumes after the call). Word 1 is argument 7. The later rows show higher addresses in the caller's stack. Arguments 1–6 arrive in registers in this example.

The same session on x86-32

At the debugger 4 — repeat on x86-32.

```
gdb -q ./callconv32 # the 32-bit build of the same program
break *foo
run
```

```
$ gdb -q ./callconv32

Reading symbols from ./callconv32...
pwndbg: loaded 179 pwndbg commands and 46 shell commands. Type pwndbg [--shell | --all] [filter .
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_ .
pwndbg> break *foo

Breakpoint 1 at 0x118d: file callconv.c, line 4.
pwndbg> run

Starting program: /tmp/ccshot/callconv32
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, foo (a=1, b=2, c=3, d=4, e=5, f=6, g=7) at callconv.c:4
4□{
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
```

Figure 5: x86-32: same program, same breakpoint.

Figure 5 displays `a=1, ..., g=7`. All seven arguments have already been placed on the stack, so GDB can read them at this stop. Compare their stack locations with the register arguments in the x86-64 example.

```
─────────────[ REGISTERS / show-flags off / show-compact-regs off ]─────────────
EAX  0x565561bd (main) ◂— lea ecx, [esp + 4]
EBX  0x56558fd8 (_GLOBAL_OFFSET_TABLE_) ◂— 0x3ee0
ECX  0xffffcdb0 ◂— 1
EDX  0xffffcdd0 —▸ 0xf7f9be34 (_GLOBAL_OFFSET_TABLE_) ◂— 0x22fd2c
EDI  0xf7ffcb60 (_rtld_global_ro) ◂— 0
ESI  0xffffce6c —▸ 0xffffd0d6 ◂— 'SHELL=/bin/bash'
EBP  0xffffcd98 ◂— 0
ESP  0xffffcd60 —▸ 0x565561ed (main+48) ◂— add esp, 0x1c
EIP  0x5655618d (foo) ◂— push ebp
```

Figure 6: x86-32: the REGISTERS panel at function entry.

In this cdecl example, the arguments are passed on the stack. Do not use the values in `EAX`, `ECX` or `EDX` to identify incoming arguments; those registers are not used for argument passing here.

```
──────────────────────[ DISASM / i386 / set emulate on ]──────────────────────
► 0x5655618d <foo>       push   ebp
  0x5655618e <foo+1>     mov    ebp, esp     EBP => 0xffffcd5c —▸ 0xffffcd98 ◂— 0
  0x56556190 <foo+3>     call   __x86.get_pc_thunk.ax       <__x86.get_pc_thunk.ax>

  0x56556195 <foo+8>     add    eax, 0x2e43
  0x5655619a <foo+13>    mov    edx, dword ptr [ebp + 8]
  0x5655619d <foo+16>    mov    eax, dword ptr [ebp + 0xc]
  0x565561a0 <foo+19>    add    edx, eax
  0x565561a2 <foo+21>    mov    eax, dword ptr [ebp + 0x10]
  0x565561a5 <foo+24>    add    edx, eax
  0x565561a7 <foo+26>    mov    eax, dword ptr [ebp + 0x14]
  0x565561aa <foo+29>    add    edx, eax
```

Figure 7: x86-32: the DISASM panel fetches the arguments from the stack.

The DISASM panel loads arguments from `[ebp+8]`, `[ebp+0xc]` and later stack slots, four bytes apart.

At the debugger 5 — the same stack dump.

```
stack 12
```

```
pwndbg> stack 12

00:0000| esp 0xffffcd60 —▸ 0x565561ed (main+48) ◂— add esp, 0x1c
01:0004|-034 0xffffcd64 ◂— 1
02:0008|-030 0xffffcd68 ◂— 2
03:000c|-02c 0xffffcd6c ◂— 3
04:0010|-028 0xffffcd70 ◂— 4
05:0014|-024 0xffffcd74 ◂— 5
06:0018|-020 0xffffcd78 ◂— 6
07:001c|-01c 0xffffcd7c ◂— 7
08:0020|-018 0xffffcd80 ◂— 0
... ↓        3 skipped
```

Figure 8: x86-32: `stack 12` at the same stop.

Compare Figures 4 and 8: the x86-64 stack contains argument 7 in eight-byte slots, while the x86-32 stack contains all seven arguments in four-byte slots. In both dumps, the return address is at the stack pointer because we stopped before the prologue.

Other useful pwndbg commands

- `regs`: show register values.
- `telescope <addr> N`: show `N` pointer-sized words and follow pointers.
- `vmmap`: show mapped memory regions and their permissions.
- `checksec`: show protections such as canaries, NX and PIE.
- `heap`: list heap chunks, their sizes and allocation states.
- `tcachebins`: show freed chunks held in the thread's tcache.
- `vis-heap-chunks`: show heap contents with chunk boundaries and colours.

For details and examples, see the pwndbg command reference. Task 2 includes a short heap-debugging example.

Task 1 – SecureAI MathOracle (40 Points)

MathOracle is SecureAI's "automated math oracle". When you start it, it prints a terminal banner, grants you 10 experience credits on your first visit, and asks for your name. Internally your session is a small structure:

```
struct account {
    char name[16];
    int balance;
};
```

The program then reads your name with a generous limit:

```
printf("Your name: ");
read(0, user.name, 0x20);
```

If your balance is zero, the program asks for feedback and prints it back to you. It then still accepts a math question, but refuses to answer without sufficient credit.

Your goal: redirect execution to a function that is never called normally and make it print `flag.txt`. You will need to preserve the stack canary and calculate the target address from the randomized executable base.

Guidance

- Review the buffer-overflow and format-string lectures. Use `%N$p` to select argument position `N`, counting from 1 after the format string. This is an argument index, not a byte offset into the stack.
- Probe positions with strings such as `%40$p %41$p %42$p`. Compare the output with the canary and saved return address inspected in GDB *in the same run.*
- After `feedback` establishes its frame, `[rbp]` contains `chat`'s saved frame pointer. Use that value to inspect `chat`'s frame. Check the disassembly to locate its canary and saved return address.
- Measure the buffer-to-canary and buffer-to-return-address distances in GDB. Preserve the canary in your second overflow so the function can return. Disassemble `main` to identify the leaked return address: it points to the instruction after a `call`. With PIE, compute `base = leaked_address - code_offset`, then `target = base + target_offset`. Use offsets relative to the executable base, not byte positions in the file.
- `read()` may return fewer bytes than requested. Sending all stages at once may let an earlier read consume later input. Send each stage after its prompt.

Report (40 Points)

1. Identify the buffer-overflow and format-string vulnerabilities. Give the source line numbers for both overflow sites and the unsafe format-string call, and explain how each contributes to the attack. (8 points)
2. Show your gdb probing process that identifies the vararg positions of the stack canary and of the leaked code pointer. Include screenshots. (8 points)
3. Which instruction in `main` does the leaked code pointer point to? Explain how you turn it into the load base and then into your target address. (8 points)
4. Draw the stack frame of the vulnerable function at the moment of the second overflow (buffer, canary, saved frame pointer, return address) and justify every byte of your payload. (10 points)
5. Provide a screenshot of a successful run in which the binary prints the flag. (6 points)

Task 2 – MindVault, the Memory Clinic (30 Points)

MindVault is a long-term memory clinic: you can store memories, read them back, and delete them. Every memory has exactly the same size and is stored as:

```
struct note {
    long len; // first 8 bytes of the allocated user area
    char data[]; // the memory itself
};
```

Reading a memory uses its length field, capped at 512 bytes:

```
long len = n->len;
if (len <= 0) { puts("(empty note)"); return; }
if (len > MAXDUMP) len = MAXDUMP;
write(1, n->data, len);
```

Deleting a memory releases it:

```
free(mem[idx]);
puts("Memory deleted. It's gone forever. Trust us.");
```

The clinic also offers a "private inference" option. It produces a classified result (a line starting with `TOP SECRET:`) that is stored somewhere on the heap, and no menu option will ever hand you that pointer.

Your goal: make the Read menu option print the classified line, using only the menu operations.

Guidance

- Review the use-after-free lifecycle slide: allocation, deallocation (dangling pointer), use. Find the line that creates the dangling pointer.
- In the course server's glibc allocator, these freed notes enter a small-block cache called `tcache`. The first 16 bytes of their user area hold a linked-list entry (offset 0) and a double-free detection key (offset 8). The link is encoded, so it may not look like a normal pointer. Inspect it with `x/4gx mem[idx]` in GDB and compare it with the `len` field that `show()` uses.
- Allocations may use fresh space or reuse freed chunks. Inspect where your two notes and the classified scratch pad are placed, and how your operation order affects whether the read can reach the classified data.
- The 512-byte cap limits how much a single read may print, but ask yourself what it does *not* limit.

- In a separate run, delete the same index twice and read the abort message. It names the allocator mechanism that detects the double free.
- Note that the sizes used for the notes and for the classified scratch pad are different; check what that means for where the scratch pad can come from.

Heap tools: a short example

- `heap`: list heap chunks, their sizes and allocation states.
- `vis-heap-chunks`: show chunk boundaries and memory contents in colour.
- `tcachebins`: show freed chunks cached for reuse.
- `malloc-chunk <addr>`: inspect the chunk whose header is at `addr`.
- `x/4gx <addr>`: read four eight-byte values (a standard GDB command).

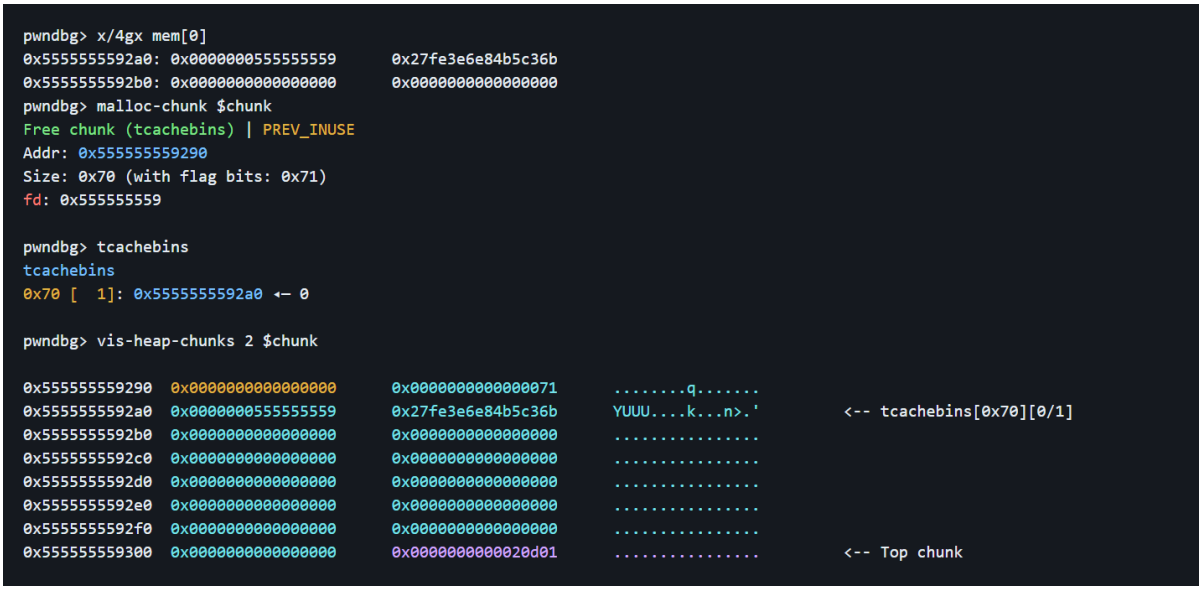


Figure 9: Example output: raw memory, chunk details, tcache and heap visualisation.

Heap visualisation command reference.

Report (30 Points)

1. Which source line creates the dangling pointer? What exactly does `mem[idx]` contain after a delete? (5 points)
2. When a note is freed into tcache, what does the allocator write into its first 16 bytes of user data? What is each field used for? (5 points)
3. Explain why reading a deleted memory can print data far beyond the chunk itself. What precisely is wrong with the 512-byte cap? (5 points)
4. Draw the relevant heap layout after each operation in your attack: show your two notes, the classified chunk when allocated, and the top chunk (remaining free space at the heap's end). Explain how your operation order lets the read reach the classified data. (5 points)
5. What happens when you delete the same index twice, and how does the allocator detect it? (5 points)
6. Provide screenshots of a successful run in which the classified line (and hence the flag) is leaked. (5 points)

Task 3 – Secure Casino (30 Points)

The house always wins – unless you refuse to play fair. Secure Casino starts you with $100. The vault at the back of the house opens only for players who (a) hold at least $1,000,000 in cash and (b) have vault access granted by their loaded profile.

The shop calculates bulk prices using 32-bit signed integers, even though the program itself is a 64-bit executable:

```
int total = item_price[id] * qty; // qty is limited to 1..65535
if (total > g_balance) { puts("Not enough cash."); return; }
g_balance -= total;
```

Your profile, including vault authorization and your name, lives in `profile.dat` and is protected by a signature `SIG` that the program verifies on every reload. The reload routine works in two phases:

```
/* check */ fopen + read + verify SIG ... fclose
            puts("Signature OK. Contacting notary server...");
            usleep(300000); // the notary server is very slow
/* use */ fopen + read again -> apply K= / B= ... fclose
```

The vault key is not for sale; vault authorization is loaded from the profile file.

Your goal: open the vault and make the program print the flag.

Guidance

- Signed integer overflow is undefined behaviour in C. For this task, analyze the 32-bit multiplication and wrap-around observed in the supplied binary.
- The receipt prints the wrapped total – watch its sign. A price that the check considers "affordable" is not necessarily positive.
- The reload routine opens the profile file twice, with a slow step in between. The program's own output tells you when the window between check and use is open; use that as the timing signal for swapping the file from a second terminal or thread.

Report (30 Points)

1. Explain the integer overflow for the quantity you chose: show the wrap-around at bit/hex level, explain why the balance check lets it pass, and include the register's printed receipt. (8 points)
2. Give at least two other quantities that each raise the balance from $100 to at least $1,000,000 through the same vulnerability. State the item and explain how you

found the quantities. (4 points)

3. Identify the check and use operations in the reload routine. Explain the approximately 300 ms delay between them and which output you used to trigger the file change. Describe your attack and include screenshots. (10 points)
4. Propose source-code patches in diff format that eliminate both vulnerabilities. Submit the diff in your report; no rebuilt binary is required. (5 points)
5. Provide a screenshot of the vault opening and printing the flag. (3 points)